

Prerequisite Challenge

READ CAREFULLY ALL STEPS

You have 48h to complete this challenge from the moment it is sent.

Objective

This challenge is **not** a test of your existing Solana or Anchor knowledge.

We're interested in understanding how you approach an unfamiliar codebase, set up a working development environment, learn independently, and communicate your understanding of a system. Feel free to use documentation, AI, tutorials, or any other resources available to you.

You will be provided with:

- An Anchor-based Vault program.
- The interface (IDL) for our registration program

Before you start

Set up your environment.

You'll need Rust, the Solana CLI, and Anchor installed. You can follow this tutorial:

<https://solana.com/developers/bootcamp/solana-crashcourse/set-up-your-solana-environment>

For getting a basic understanding for how to use anchor locally:

<https://www.anchor-lang.com/docs/quickstart/local>

Look at some basic Solana concepts.

You don't need to know these in depth to start (we will get to that once the cohort starts), but knowing what the words mean will make the code much easier to read:

- **Program.** A smart contract on Solana. The Vault program is one.
- **Anchor.** A Rust framework that removes much of the boilerplate involved in writing Solana programs.
- **Account.** Where Solana stores data. Programs themselves are stateless, so the state lives in accounts that get passed into each instruction.
- **Instruction.** A single callable action in a program, such as deposit or withdraw.
- **PDA (Program Derived Address).** A PDA is used to enable our program to "sign" transactions with a Public Key derived from some kind of deterministic seed. This is then combined with an additional "bump" which is a single additional byte that is generated to

"bump" this Public Key off the elliptic curve. This means that there is no matching Private Key for this Public Key, as if there were a matching private key and someone happened to possess it, they would be able to sign on behalf of the program, creating security concerns.

- **CPI (Cross-Program Invocation)**. One program calling another program from inside its own instruction. That's what Task 2 is about.
 - **IDL (Interface Definition Language)**. Similar to the concept of ABI in other ecosystems, an IDL specifies a program's public interface. Though not mandatory, most programs on Solana do have an IDL, and it is the primary way we typically interact with programs on Solana. It defines a Solana program's account structures, instructions, and error codes. IDLs are .json files, so they can be used to generate client-side code, such as Typescript type definitions, for ease of use.
-

Task 1: Understand the Vault Program

Github link : [Vault program](#)

The repository contains the Anchor vault program along with tests implemented in both Litesvm (Rust) and TypeScript. To interact with the registration program on devnet in the next task, we will focus on the TypeScript tests.

Read through the Vault program that is given to you and develop a high-level understanding of how it works.

By the end of this task, you should understand:

- The purpose of each instruction.
- The accounts involved.
- How program state changes over time.
- The overall flow of the application.

You'll notice that the *withdraw* function includes accounts such as *Application_account* and *Application_program*, which are currently unused and not needed for Withdraw instruction. This leads directly to Task 2.

Task 2: Extend the Program

Modify the Vault program by extending the **withdraw** instruction to perform a Cross-Program Invocation (CPI) to our registration program. All the necessary functions and Account struct needed to perform the CPI to call initialize has already been imported.

The registration program records:

- GitHub username

Deploy the modified program and run *anchor test*.

It's important that you deploy the program and not call the program we have deployed, because we will verify that the CPI is not done using our program or a direct call to the registration program. Keep in mind that you can do only one registration per wallet.

Success criteria: After a successful withdrawal, the registration program should successfully record your details.

Task 3: Create an Architecture Diagram

Create a simple architecture diagram that explains how the Vault program works. You can use any tool of your choice, including Excalidraw, draw.io, or others.

Assume your audience is a new engineer joining the project. Your diagram should clearly communicate the important accounts, instructions, and overall flow of the program.

Task 4: Record a Video

Record a video (**maximum 3 minutes**) explaining your architecture diagram. Use any tool you like, and upload it to Youtube, make sure we can access it and captions are available. Upload the link in the form shared below.

Focus on explaining the overall design and flow of the program rather than walking through the code line by line. Resolution and production quality don't matter, so record on whatever device you have, including your phone, as long as we can hear you clearly. What we're really evaluating is how well you communicate your understanding of the task.

Deliverables

Submit the following:

- Modified Vault program containing the verification CPI, in a public Github repository.
- Architecture diagram.
- Video (maximum 3 minutes).

The submission is complete by completing this form with the required deliverables:

<https://forms.gle/zGPY8svmPdMQg3rG9>

Evaluation Criteria

We'll be evaluating:

- Your ability to learn independently.
 - Your ability to follow instructions and complete the task.
 - Your ability to understand and extend an existing codebase.
 - Your ability to communicate technical concepts clearly.
-

Getting help

Use any resource you like, including AI, video tutorials, and documentation. There's a list of useful resources below.

You can also ask questions in Discord. You're part of a community now, and it's a supportive one, so help each other out. That's a big part of how this cohort works. If you are not already on our Discord server, here is a link to join: <https://discord.gg/fPRYmMC6E>

One rule: don't ask for or share solutions. Do ask about what's blocking you, a concept you're missing, or a step that isn't making sense. We want to see how you think, not how well the group converged on one answer.

Useful resources

- Solana docs, start here: <https://solana.com/docs>
- Solana core concepts, accounts, PDAs, CPIs: <https://solana.com/docs/core>
- Anchor documentation: <https://www.anchor-lang.com/docs>
- Solana Stack Exchange, for specific questions: <https://solana.stackexchange.com>